

Building a Multiclass Classification Model in PyTorch

by [Adrian Tam](#) on [April 8, 2023](#) in [Deep Learning with PyTorch](#)  12

 Share  Share  Post

The PyTorch library is for deep learning. Some applications of deep learning models are used to solve regression or classification problems. In this tutorial, you will discover how to use PyTorch to develop and evaluate neural network models for multi-class classification problems.

After completing this step-by-step tutorial, you will know:

- How to load data from CSV and make it available to PyTorch
- How to prepare multi-class classification data for modeling with neural networks
- How to use cross validation to evaluate a PyTorch neural network model

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Building a multiclass classification model in PyTorch
Photo by Cheung Yin. Some rights reserved.

Problem Description

In this tutorial, you will use a standard machine learning dataset called the iris flowers dataset. It is a well-studied dataset and good for practicing machine learning. It has four input variables; all are numeric and length measurements in centimeters. Therefore, they are on a similar scale. Each data sample describes the properties of an observed iris flower. Its goal is to use the measurements (input features) to classify the iris species (output label).

There are three iris species in the dataset. Therefore, it is a multi-class classification problem. Multi-class classification problems are special because they require special handling to specify a class.

This dataset came from Sir Ronald Fisher, the father of modern statistics. It is the best-known dataset for pattern recognition, and you can achieve a model accuracy in the range of 95% to 97%. You can make this your target in developing the deep learning model.

You can download the [iris flowers dataset](#) from the UCI Machine Learning repository and place it in your current working directory with the filename "iris.csv". You can also download the dataset [here](#).

Load the Dataset

There are multiple ways to read a CSV file. The easiest way is probably to use a pandas library. After reading the dataset, you want to split it into features and labels as you need to further process the labels before use. Unlike NumPy or PyTorch tensors, a pandas DataFrame can do slicing by indices only through `iloc`:

```
1 import pandas as pd
2 data = pd.read_csv("iris.csv", header=None)
3 X = data.iloc[:, 0:4]
4 y = data.iloc[:, 4:]
```

Now, you have loaded the dataset and split the attributes (i.e., input features, columns in the DataFrame) as X and the output variables (i.e., species labels) as a single-column DataFrame y.

Encode the Categorical Variable

The species labels are strings, but you want them in numbers. It is because numerical data are easier to use. In this dataset, the three class labels are Iris-setosa, Iris-versicolor, and Iris-virginica. One way to convert these labels into a number (i.e., encode them) is simply to assign an integer value such as 0, 1, or 2 to replace these labels. But there is a problem: You do not want the model to think that Iris-virginica is the sum of Iris-setosa and Iris-versicolor. In fact, in statistics, there are levels of measurement:

- Nominal numbers: Those numbers are, in fact, names. Operations on them do not make sense
- Ordinal numbers: They are orders of something. Comparing for greater or less than makes sense, but addition or subtraction does not
- Interval numbers: They are measurements, such as the year today, so that subtraction bears meaning (e.g., how old are you), but the zero value is arbitrary and not special
- Ratio numbers: Like interval, but the zero is meaningful, such as the measure of length or time. In this case, both subtraction and division bear meaning, and you can say that something is twice as long.

The encoded label is nominal. You do not want to mistake it as interval or ratio data, but your model would not know. One way to avoid such a mistake is with **one-hot encoding**, which instead of converting a label into an integer, converts a label into a **one-hot vector**. A one-hot vector is a vector of integers, but only one of them is 1, and the rest are all zero. In this case, you convert the labels into the following:

```
1 Iris-setosa      1  0  0
2 Iris-versicolor  0  1  0
3 Iris-virginica   0  0  1
```

The above is a one-hot encoded binary matrix. You don't need to create it manually. You can encode the strings consistently to integers using the scikit-learn class `LabelEncoder` or into one-hot encoding vectors using the class `OneHotEncoder`:

```
1 from sklearn.preprocessing import OneHotEncoder
2 ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit(y)
3 print(ohe.categories_)
4
5 y = ohe.transform(y)
6 print(y)
```

From these, you can see the `OneHotEncoder` learned the three categories:

```
1 [array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'], dtype=object)]
```

Then the string labels are transformed into one-hot vectors like the following:

```
1 [[1.  0.  0.]
2  [1.  0.  0.]
3  [1.  0.  0.]
4  ...
5  [0.  0.  1.]
6  [0.  0.  1.]
7  [0.  0.  1.]]
```

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Define the Neural Network Model

Now you need to have a model that can take the input and predict the output, ideally in the form of one-hot vectors. There is no science behind the design of a perfect neural network model. But know one thing—it has to take in a vector of 4 features and output a vector of 3 values. The 4 features correspond to what you have in the dataset. The 3-value output is because we know the one-hot vector has 3 elements. Anything can be in between, known as the “hidden layers,” since they are neither input nor output.

The simplest is to have only one hidden layer. Let’s make one like this:

1 [4 inputs] -> [8 hidden neurons] -> [3 outputs]

Such a design is called the network topology. You should use a “softmax” activation at the output layer. In the formula, it means:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}}$$

This normalizes the values (z_1, z_2, z_3) and applies a non-linear function such that the sum of all 3 outputs will be 1, and each of them is in the range of 0 to 1. This makes the output look like a vector of probabilities. The use of the softmax function at the output is the signature of a multi-class classification model. But in PyTorch, you can skip this if you combine it with an appropriate loss function.

In PyTorch, you can build such a model as follows:

```
1 import torch
2 import torch.nn as nn
3
4 class Multiclass(nn.Module):
5     def __init__(self):
6         super().__init__()
7         self.hidden = nn.Linear(4, 8)
8         self.act = nn.ReLU()
9         self.output = nn.Linear(8, 3)
10
11     def forward(self, x):
12         x = self.act(self.hidden(x))
13         x = self.output(x)
14         return x
15
16 model = Multiclass()
```

The output of this model is the “weight” of the three classes. Ideally, the model output will be such that only one of the elements is positive infinity, and the rest are negative infinity, making an extreme contrast with absolute confidence to which of the three classes the input features belong. In an imperfect situation, as always happens, you can expect a good model to tell you that one of the values is very positive, and the others are very negative. Or if you transform these values using a sigmoid function or softmax function, one is very close to 1, and the others are very close to 0.

In this case, the loss metric for the output can simply be measuring how close the output is to the one-hot vector you transformed from the label. But usually, in multi-class classification, you use categorical cross entropy as the loss metric. In the formula, it is:

$$H(p, q) = - \sum_x p(x) \log q(x)$$

This means, given the real probability vector $p(x)$ and the predicted probability vector $q(x)$, the similarity is the sum of the product of $p(x)$ and $\log q(x)$ for each element x . The one-hot vector is considered as the probability vector $p(x)$, and the model output is $q(x)$. Since it is a one-hot vector, only the actual class has $p(x) = 1$, and the other classes have $p(x) = 0$. The sum above is essentially $-\log q(x)$ of the actual class x . The value will be 0 when $q(x) = 1$, and as $q(x)$ approaches 0 (the minimum value softmax can produce), $-\log q(x)$ approaches infinity.

Below is how you can define the loss metric. The CrossEntropyLoss function in PyTorch combines the softmax function with the cross entropy calculation, so you don’t need any activation function at the output layer of your model. You also need an optimizer, and Adam is chosen below.

```
1 import torch.optim as optim
2
3 loss_fn = nn.CrossEntropyLoss()
4 optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Note that when you define the optimizer, you need to tell it the model parameters as well since these are what the optimizer is going to update.

Now you need to run the training loop to train your model. Minimally, you need to put three steps in the loop: a forward pass, a backward pass, and the weight update. The forward pass provides the input to the model and takes the output. The backward pass starts with the loss metric, which is based on the model output, and propagates the gradient back to the input. The weight update is based on the gradient used to update the weights.

A minimal training loop can be implemented using a for-loop. But you can make use of tqdm to create a progress bar visualization:

```
1 import tqdm
2
3 # convert pandas DataFrame (X) and numpy array (y) into PyTorch tensors
4 X = torch.tensor(X.values, dtype=torch.float32)
5 y = torch.tensor(y, dtype=torch.float32)
6
7 # training parameters
8 n_epochs = 200
9 batch_size = 5
10 batches_per_epoch = len(X) // batch_size
11
12 for epoch in range(n_epochs):
13     with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
14         bar.set_description(f"Epoch {epoch}")
15         for i in bar:
16             # take a batch
17             start = i * batch_size
18             X_batch = X[start:start+batch_size]
19             y_batch = y[start:start+batch_size]
20             # forward pass
21             y_pred = model(X_batch)
22             loss = loss_fn(y_pred, y_batch)
23             # backward pass
24             optimizer.zero_grad()
25             loss.backward()
26             # update weights
27             optimizer.step()
```

Benchmark the Model

The goal of a model is never to match the dataset per se. The reason you want to build a machine learning model is to prepare for the data you will encounter in the future, which is unseen yet. How can you know the model can do that? You need a test set. It is a dataset that is the same structure as the one used in training but is separate. So it is like unseen data from the training process, and you can take that as a benchmark. This technique of evaluating a model is called **cross validation**.

Usually, you do not add a test set but split the data set you obtained into a training set and a test set. Then you use the test set to evaluate the model at the end. Such a benchmark serves another purpose: You do not want your model to overfit. It means the model learned too much about the training set and failed to generalize. If this happens, you will see that the model does not give a good result on the test set.

Splitting the data into a training set and a test set can be easily done with scikit-learn. The workflow from loading data to one-hot encoding and splitting is as follows:

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import OneHotEncoder
4
5 # read data and apply one-hot encoding
6 data = pd.read_csv("iris.csv", header=None)
7 X = data.iloc[:, 0:4]
8 y = data.iloc[:, 4:]
9 ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit(y)
10 y = ohe.transform(y)
11
12 # convert pandas DataFrame (X) and numpy array (y) into PyTorch tensors
13 X = torch.tensor(X.values, dtype=torch.float32)
14 y = torch.tensor(y, dtype=torch.float32)
15
16 # split
17 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
```

The arguments `train_size=0.7` with `shuffle=True` means to randomly select 70% of the samples in the dataset into the training set, while the rest will become the test set.

Once you do that, you need to modify the training loop to use the training set in training and use the test set at the end of each epoch for benchmarking:

```
1 n_epochs = 200
2 batch_size = 5
3 batches_per_epoch = len(X_train) // batch_size
4
5 for epoch in range(n_epochs):
6     with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
7         bar.set_description(f"Epoch {epoch}")
8         for i in bar:
9             # take a batch
10             start = i * batch_size
11             X_batch = X_train[start:start+batch_size]
12             y_batch = y_train[start:start+batch_size]
13             # forward pass
```

```

14     y_pred = model(X_batch)
15     loss = loss_fn(y_pred, y_batch)
16     # backward pass
17     optimizer.zero_grad()
18     loss.backward()
19     # update weights
20     optimizer.step()
21     y_pred = model(X_test)
22     ce = loss_fn(y_pred, y_test)
23     acc = (torch.argmax(y_pred, 1) == torch.argmax(y_test, 1)).float().mean()
24     print(f"Epoch {epoch} validation: Cross-entropy={float(ce)}, Accuracy={float(acc)}")

```

That's almost everything you need to finish a deep learning model in PyTorch. But you may also want to do a bit more: Firstly, after all the training epochs, you may want to roll back the model to the best you ever achieved rather than the last one. Secondly, you may want to produce a graph to visualize how the cross entropy and accuracy progressed.

It is not difficult to do. In the training loop, you keep track of the accuracy on the test set and keep a copy of the model whenever this accuracy is higher. And at the same time, remember the metrics you calculated in a list. Then at the end of the training loop, you restore the best model you ever saw and plot the metrics as a time series.

In code, this is how the training loop is modified into:

```

1  import copy
2  import tqdm
3  import numpy as np
4
5  n_epochs = 200
6  batch_size = 5
7  batches_per_epoch = len(X_train) // batch_size
8
9  best_acc = - np.inf # init to negative infinity
10 best_weights = None
11 train_loss_hist = []
12 train_acc_hist = []
13 test_loss_hist = []
14 test_acc_hist = []
15
16 for epoch in range(n_epochs):
17     epoch_loss = []
18     epoch_acc = []
19     # set model in training mode and run through each batch
20     model.train()
21     with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
22         bar.set_description(f"Epoch {epoch}")
23         for i in bar:
24             # take a batch
25             start = i * batch_size
26             X_batch = X_train[start:start+batch_size]
27             y_batch = y_train[start:start+batch_size]
28             # forward pass
29             y_pred = model(X_batch)
30             loss = loss_fn(y_pred, y_batch)
31             # backward pass
32             optimizer.zero_grad()
33             loss.backward()
34             # update weights
35             optimizer.step()
36             # compute and store metrics
37             acc = (torch.argmax(y_pred, 1) == torch.argmax(y_batch, 1)).float().mean()
38             epoch_loss.append(float(loss))
39             epoch_acc.append(float(acc))
40             bar.set_postfix(
41                 loss=float(loss),
42                 acc=float(acc)
43             )
44     # set model in evaluation mode and run through the test set
45     model.eval()
46     y_pred = model(X_test)
47     ce = loss_fn(y_pred, y_test)
48     acc = (torch.argmax(y_pred, 1) == torch.argmax(y_test, 1)).float().mean()
49     ce = float(ce)
50     acc = float(acc)
51     train_loss_hist.append(np.mean(epoch_loss))
52     train_acc_hist.append(np.mean(epoch_acc))
53     test_loss_hist.append(ce)
54     test_acc_hist.append(acc)
55     if acc > best_acc:
56         best_acc = acc
57         best_weights = copy.deepcopy(model.state_dict())
58     print(f"Epoch {epoch} validation: Cross-entropy={ce}, Accuracy={acc}")
59
60 model.load_state_dict(best_weights)

```

When you run it, you will see something like this, where you see the accuracy and cross entropy loss at each epoch:

```

1 Epoch 0: 100%|████████████████████| 21/21 [00:00<00:00, 850.82batch/s, acc=0.4, loss=1.23]
2 Epoch 0 validation: Cross-entropy=1.10, Accuracy=60.0%
3 Epoch 1: 100%|████████████████████| 21/21 [00:00<00:00, 3155.53batch/s, acc=0.4, loss=1.24]
4 Epoch 1 validation: Cross-entropy=1.08, Accuracy=57.8%
5 Epoch 2: 100%|████████████████████| 21/21 [00:00<00:00, 3489.58batch/s, acc=0.4, loss=1.24]
6 Epoch 2 validation: Cross-entropy=1.07, Accuracy=60.0%
7 Epoch 3: 100%|████████████████████| 21/21 [00:00<00:00, 3312.79batch/s, acc=0.4, loss=1.22]

```

```

8 Epoch 3 validation: Cross-entropy=1.06, Accuracy=62.2%
9 ...
10 Epoch 197: 100%|██████████| 21/21 [00:00<00:00, 3529.57batch/s, acc=1, loss=0.563]
11 Epoch 197 validation: Cross-entropy=0.61, Accuracy=97.8%
12 Epoch 198: 100%|██████████| 21/21 [00:00<00:00, 3479.10batch/s, acc=1, loss=0.563]
13 Epoch 198 validation: Cross-entropy=0.61, Accuracy=97.8%
14 Epoch 199: 100%|██████████| 21/21 [00:00<00:00, 3337.52batch/s, acc=1, loss=0.563]
15 Epoch 199 validation: Cross-entropy=0.61, Accuracy=97.8%

```

You added more lines into the training loop but for a good reason. It is a good practice to switch the model between training mode and evaluation mode when you are switching between the training set and test set. In this particular model, nothing is changed. But for some other models, it will affect the model's behavior.

You collected metrics in Python lists. You need to be careful to convert the PyTorch tensor (even if it is a scalar value) into a Python float. The purpose of this conversion is to make a copy of the number so that PyTorch will not silently mutate it (e.g., through the optimizer).

After each epoch, you calculated the accuracy based on the test set, and the model weight is stored if you saw the accuracy is higher. However, when you take out the model weight, you should make a deep copy; otherwise, you lost them when the model changes its weights in the next epoch.

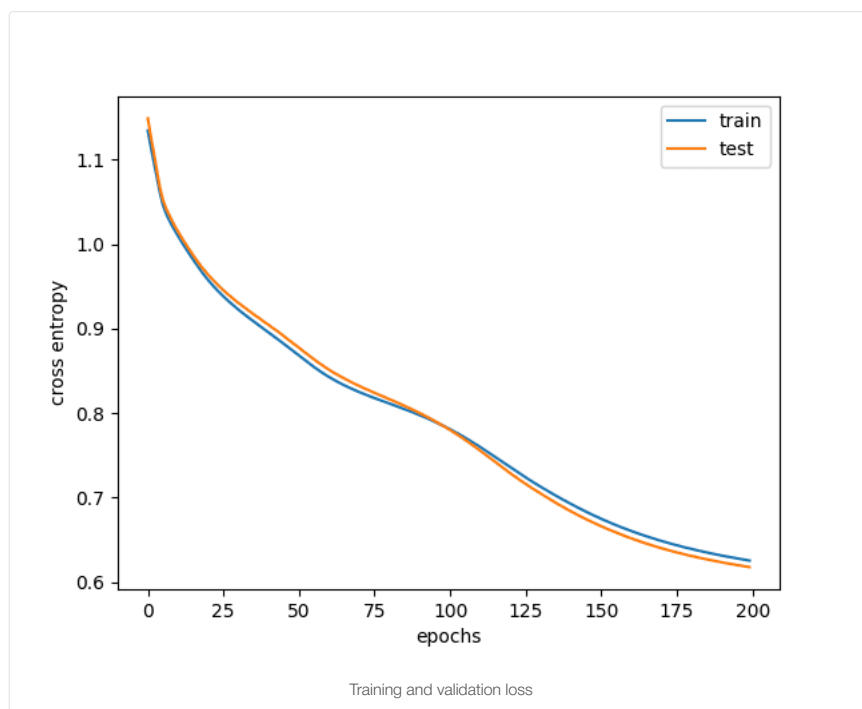
Finally, you can plot the loss and accuracy across epochs using matplotlib as follows:

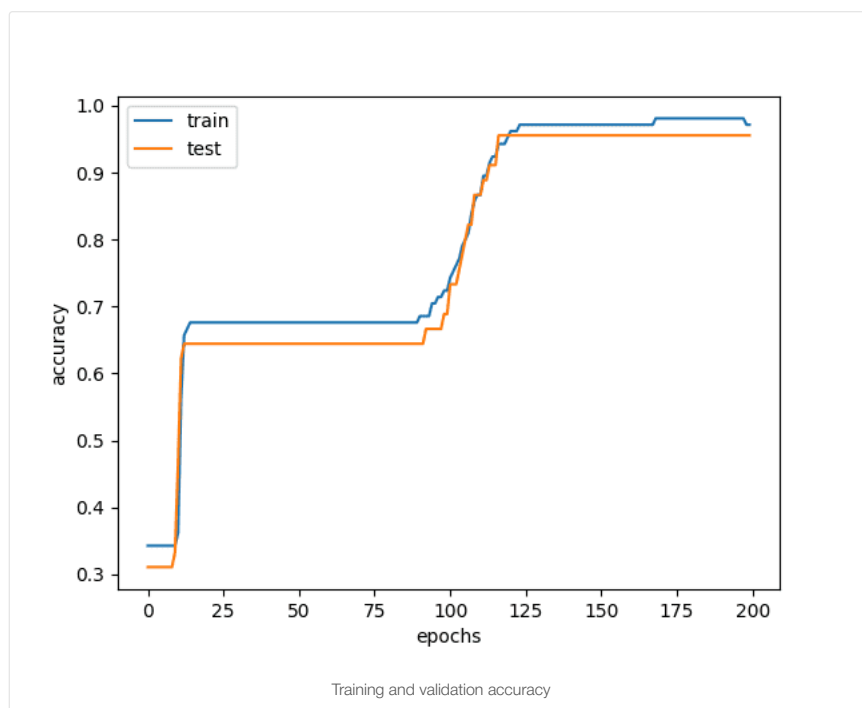
```

1 import matplotlib.pyplot as plt
2
3 plt.plot(train_loss_hist, label="train")
4 plt.plot(test_loss_hist, label="test")
5 plt.xlabel("epochs")
6 plt.ylabel("cross entropy")
7 plt.legend()
8 plt.show()
9
10 plt.plot(train_acc_hist, label="train")
11 plt.plot(test_acc_hist, label="test")
12 plt.xlabel("epochs")
13 plt.ylabel("accuracy")
14 plt.legend()
15 plt.show()

```

A typical result is as follows:





You can see from the graph that at the beginning, both training and test accuracy are very low. This is when your model is underfitting and is performing horribly. As you keep training the model, the accuracy increases, and the cross entropy loss decreases. But at a certain point, the training accuracy is higher than the test accuracy, and in fact, even when the training accuracy improves, the test accuracy flattened or even lowered. This is when the model overfitted, and you do not want to use such a model. That's why you want to keep track of the test accuracy and restore the model weight to the best result based on the test set.

Complete Example

Putting everything together, the following is the complete code:

```
1 import copy
2
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import pandas as pd
6 import torch
7 import torch.nn as nn
8 import torch.optim as optim
9 import tqdm
10 from sklearn.model_selection import train_test_split
11 from sklearn.preprocessing import OneHotEncoder
12
13 # read data and apply one-hot encoding
14 data = pd.read_csv("iris.csv", header=None)
15 X = data.iloc[:, 0:4]
16 y = data.iloc[:, 4:]
17 ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit(y)
18 y = ohe.transform(y)
19
20 # convert pandas DataFrame (X) and numpy array (y) into PyTorch tensors
21 X = torch.tensor(X.values, dtype=torch.float32)
22 y = torch.tensor(y, dtype=torch.float32)
23
24 # split
25 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
26
27 class Multiclass(nn.Module):
28     def __init__(self):
29         super().__init__()
30         self.hidden = nn.Linear(4, 8)
31         self.act = nn.ReLU()
32         self.output = nn.Linear(8, 3)
33
34     def forward(self, x):
35         x = self.act(self.hidden(x))
36         x = self.output(x)
37         return x
38
39 # loss metric and optimizer
40 model = Multiclass()
41 loss_fn = nn.CrossEntropyLoss()
42 optimizer = optim.Adam(model.parameters(), lr=0.001)
43
44 # prepare model and training parameters
45 n_epochs = 200
46 batch_size = 5
```

```

47 batches_per_epoch = len(X_train) // batch_size
48
49 best_acc = - np.inf # init to negative infinity
50 best_weights = None
51 train_loss_hist = []
52 train_acc_hist = []
53 test_loss_hist = []
54 test_acc_hist = []
55
56 # training loop
57 for epoch in range(n_epochs):
58     epoch_loss = []
59     epoch_acc = []
60     # set model in training mode and run through each batch
61     model.train()
62     with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
63         bar.set_description(f"Epoch {epoch}")
64         for i in bar:
65             # take a batch
66             start = i * batch_size
67             X_batch = X_train[start:start+batch_size]
68             y_batch = y_train[start:start+batch_size]
69             # forward pass
70             y_pred = model(X_batch)
71             loss = loss_fn(y_pred, y_batch)
72             # backward pass
73             optimizer.zero_grad()
74             loss.backward()
75             # update weights
76             optimizer.step()
77             # compute and store metrics
78             acc = (torch.argmax(y_pred, 1) == torch.argmax(y_batch, 1)).float().mean()
79             epoch_loss.append(float(loss))
80             epoch_acc.append(float(acc))
81             bar.set_postfix(
82                 loss=float(loss),
83                 acc=float(acc)
84             )
85         # set model in evaluation mode and run through the test set
86         model.eval()
87         y_pred = model(X_test)
88         ce = loss_fn(y_pred, y_test)
89         acc = (torch.argmax(y_pred, 1) == torch.argmax(y_test, 1)).float().mean()
90         ce = float(ce)
91         acc = float(acc)
92         train_loss_hist.append(np.mean(epoch_loss))
93         train_acc_hist.append(np.mean(epoch_acc))
94         test_loss_hist.append(ce)
95         test_acc_hist.append(acc)
96         if acc > best_acc:
97             best_acc = acc
98             best_weights = copy.deepcopy(model.state_dict())
99         print(f"Epoch {epoch} validation: Cross-entropy={ce:.2f}, Accuracy={acc*100:.1f}%")
100
101 # Restore best model
102 model.load_state_dict(best_weights)
103
104 # Plot the loss and accuracy
105 plt.plot(train_loss_hist, label="train")
106 plt.plot(test_loss_hist, label="test")
107 plt.xlabel("epochs")
108 plt.ylabel("cross entropy")
109 plt.legend()
110 plt.show()
111
112 plt.plot(train_acc_hist, label="train")
113 plt.plot(test_acc_hist, label="test")
114 plt.xlabel("epochs")
115 plt.ylabel("accuracy")
116 plt.legend()
117 plt.show()

```

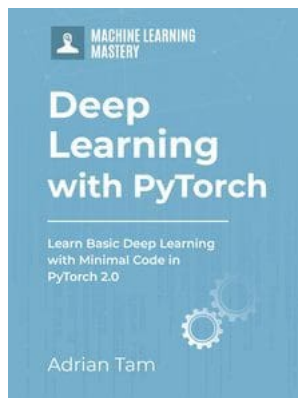
Summary

In this post, you discovered how to develop and evaluate a neural network for multi-class classification using PyTorch.

By completing this tutorial, you learned:

- How to load data and convert them to PyTorch tensors
- How to prepare multi-class classification data for modeling using one-hot encoding
- How to define a deep learning model with PyTorch for multi-class classification
- How to measure the likeness of model output to the expected result of a multi-class classification
- How to run a training loop on a PyTorch model and collect evaluation metrics

Get Started on Deep Learning with PyTorch!



Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

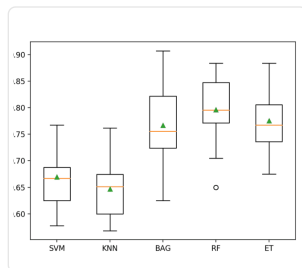
SEE WHAT'S INSIDE

Share

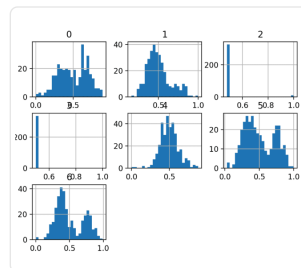
Share

Post

More On This Topic



Imbalanced Multiclass Classification with the Glass...



Imbalanced Multiclass Classification with the E.coli Dataset



Building a Binary Classification Model in PyTorch



Building a Regression Model in PyTorch



Building a Logistic Regression Classifier in PyTorch



Building a Softmax Classifier for Images in PyTorch



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
[View all posts by Adrian Tam →](#)

< [How to Evaluate the Performance of PyTorch Models](#)

[Building a Binary Classification Model in PyTorch](#) >

12 Responses to *Building a Multiclass Classification Model in PyTorch*

**Alberto** February 3, 2023 at 8:01 am #

REPLY ↩

Hi Adrian:

Nice tutorial. Thank you .

In the line

`one = OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit(y)`

the 'sparse_output' param does not work for me. I had to change it to 'sparse'

best

**Adrian Tam** February 3, 2023 at 8:51 am #

REPLY ↩

Glad you like it. The reason you saw a problem is because of the scikit-learn version you used. In version 1.2, the parameter became `sparse_output` while the earlier version says `sparse`.

**James Carmichael** February 3, 2023 at 8:53 am #

REPLY ↩

Hi Alberto...You are very welcome! Thank you for the feedback!

**Juan** April 7, 2023 at 6:52 pm #

REPLY ↩

How do you make predictions with the trained model?

**James Carmichael** April 8, 2023 at 9:08 am #

REPLY ↩

Hi Juan...The following may be of interest to you:

<https://machinelearningmastery.com/how-to-make-classification-and-regression-predictions-for-deep-learning-models-in-keras/>**Anthony** July 24, 2023 at 10:38 am #

REPLY ↩

Thank you. I'm a pytorch newbie and your tutorials are easy to understand.

**James Carmichael** July 25, 2023 at 8:32 am #

REPLY ↩

You are welcome Anthony! We are pleased to know that you are making progress with our tutorials.

**SFLam** March 17, 2024 at 10:15 am #

REPLY ↩

I got the following error when trying to run the code under Colab (all previous codes up to that point were fine):

File "", line 3

`loss_fn = nn.CrossEntropyLoss()`

^

SyntaxError: invalid syntax ^

**James Carmichael** March 18, 2024 at 6:23 am #

REPLY ↩

Hi SFLam...Did you copy and paste the code or type it in?

**SFLam** March 17, 2024 at 10:32 am #

REPLY ↩

The potential error lies here:

```
import torch.optim as optim

loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```



kostas March 31, 2024 at 11:24 am #

REPLY ↩

Hi, I got the following error when trying to run the code under VS Code.

```
Epoch 0: : 0batch [00:00, ?batch/s]
Traceback (most recent call last):

88 line ce = loss_fn(y_pred, y_test)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

RuntimeError: 0D or 1D target tensor expected, multi-target not supported
```



James Carmichael April 1, 2024 at 9:10 am #

REPLY ↩

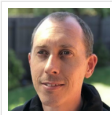
Hi kostas...Did you copy and paste the code or type it in? Also, have you tried your code in Google Colab?

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT



Welcome!
I'm *Jason Brownlee* PhD
and I **help developers** get results with **machine learning**.
[Read more](#)

Never miss a tutorial:



Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Building a Regression Model in PyTorch

Loving the Tutorials?

The [Deep Learning with PyTorch EBook](#) is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. Visit our corporate website to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved